

Oak GTM Automation - Setup Guide

Ten n8n workflows that take a booth badge scan and turn it into a routed, enriched, CRM-synced lead with a Slack card a rep can act on, in about two seconds of scanner-visible latency.

This guide covers what the workflows are, how to stand them up, and how to configure them once they are running.

The workflows

File	Role	Trigger
00 Setup & Seed	Creates the Data Tables, policy, competitors, test payloads, HubSpot properties and Slack channels	Manual, run once
01 Intake & Decision	The orchestrator. The booth webhook lands here; classification, scoring and routing are decided here	Webhook
02 HubSpot Sync	The only place the automated pipeline writes to the CRM	Called by 01
02A CRM Context	The read half of that pair. Every call is a GET or a search	Called by 01
03 Slack Alerts	Builds and delivers the card, threads repeat visits under the first alert	Called by 01
04 Company & Person Enrichment	Homepage read plus gated public search; headcount and identity evidence	Called by 01
05 AI Lead Reasoning	Bounded LLM step: persona read on ambiguous titles, opening question, listen-fors	Called by 01
06 Slack Actions	Twelve rep buttons and four modals. Writes to HubSpot only on a human tap	Slack interactivity webhook
07 Task Test Harness	Replays payloads through the live webhook and reports PASS/FAIL per scan	Manual
99 Failure Handler	Shared error workflow that turns a failure into a plain-language Slack report	n8n error trigger

Only 01 and 06 are reachable from outside. 00 and 07 are run by hand. 99 has no caller: n8n invokes it when a workflow that names it fails.

What you need first

n8n	Cloud, or self-hosted with the @brave/n8n-nodes-brave-search community package installed
HubSpot	A private app or OAuth connection with contacts, companies, deals, notes and tasks scopes
Slack	An app with chat:write , channels:manage , groups:write , reactions:write , users:read , users:read.email , im:write , plus Interactivity enabled
Gemini and Brave	On n8n Cloud these run on gateway credits with no credential to create. Self-hosted needs an API key for each

Setup

1. Import the workflows

Import all ten JSON files into the same n8n project.

2. Create the credentials

Three, and note that two are Slack of different types:

Credential	Type	Used by
HubSpot	hubspotOAuth2Api	00 , 02 , 02A , 06
Slack	slackOAuth2Api	00 , 99
Slack app	slackApi	03 , 06

3. Set the n8n variables

Variable	Used by	Effect
<code>OAK_WEBHOOK_SECRET</code>	<code>01</code> verifies it, <code>07</code> sends it	Turns on webhook authentication. Until it is set, every scan is accepted and the decision records <code>auth_mode: open_no_secret_configured</code>
<code>OAK_SLACK_SIGNING_SECRET</code>	<code>06</code>	Verifies Slack's HMAC. Required for the buttons to work
<code>OAK_SLACK_ACTIONS_ENABLED</code>	<code>03</code> , <code>06</code>	Set to <code>true</code> to render the action buttons on cards
<code>OAK_BOOTH_WEBHOOK_URL</code>	<code>07</code>	Your <code>01</code> production webhook URL, <code>https://<your-instance>/webhook/oak-event/scan</code> . The test harness sends its scans here
<code>OAK_TEST_SUITE</code>	<code>07</code>	Optional. Which test rows to run: unset for the enabled rows, <code>task</code> for the four task leads, <code>all</code> for every row, or any suite name

4. Run `00 Setup & Seed`

Open it and run it once. It creates:

- six Data Tables - `oak_policies` , `oak_competitors` , `oak_deliveries` , `oak_encounters` , `oak_enrichment_cache` , `oak_test_payloads`
- the active policy row (`oak-event-v5`)
- 17 competitor identities
- 20 test payloads across nine suites
- the `oak_*` HubSpot contact properties
- six Slack channels - `#booth-hot` , `#booth-matched` , `#booth-review` , `#competitive-intel` (private), `#booth-out-of-scope` , `#oak-revops`

Every workflow finds these tables by name, so nothing needs re-pointing afterwards. It is safe to run again: anything that already exists is counted as provisioned.

5. Connect the five sub-workflows in `01`

Open `01` and re-pick the target workflow on each of these nodes from the dropdown:

Node in <code>01</code>	Calls
<code>Read CRM Context</code>	<code>02A</code>
<code>Enrich Company</code>	<code>04</code>
<code>Reason with AI</code>	<code>05</code>
<code>Sync to HubSpot</code>	<code>02</code>
<code>Notify Slack</code>	<code>03</code>

Sub-workflow references are stored as IDs, so they point at the source instance until you re-pick them. Do this before the first scan.

6. Set the shared error workflow

On **01**, **02**, **02A**, **03**, **04**, **05**, **06** and **07**: **Settings -> Error workflow -> 99 Failure Handler**. Failures then post to **#oak-revops** naming the stage, the likely cause and a link to the execution.

7. Invite the Slack app to the channels

Invite the app to all six channels. **#competitive-intel** is private, so a member has to invite it there.

8. Point Slack at 06

In your Slack app settings, set the Interactivity request URL to the production webhook URL of **06**'s **Receive Slack Action** node.

9. Activate

Activate **01** through **07** and **99**. Leave **00** inactive - it is a manual setup workflow.

Verify

Run **07 Run Task Test Suite**. It replays every enabled row of **oak_test_payloads** through the live booth webhook and scores each scan from stored state.

Expected for the four task leads:

Lead	Classification	Channel
Rachel Green	tier_1	#booth-hot
Hiroshi Tanaka	matched	#booth-matched
David Miller	out_of_scope	#booth-out-of-scope
Sarah Connor	competitor_intel	#competitive-intel

A PASS asserts, in order: the webhook accepted the scan, a decision row was stored, the ledger reached **complete**, Slack delivered the alert, and then the expected classification, persona and channel.

Choose which scans run with the **OAK_TEST_SUITE** variable:

Value	Runs
<code>task</code>	The four task leads
<code>all</code>	Every row in <code>oak_test_payloads</code>
a suite name, e.g. <code>edge-cases</code>	That suite only
unset	The rows marked <code>enabled</code> in the table

The four task leads are enabled by default. The remaining rows cover edge cases, validation, boundaries, caching, security, personas, triggers, competitors and returning visitors; `docs/test-cases.md` lists every case and its expected outcome. A suite name that matches no row stops the run and lists the suites that exist.

Configuring it

Almost everything is a row in `oak_policies`, read at runtime. Edit `document_json` on the active row and the next scan follows it - no redeploy, no workflow edit:

Block	Controls
<code>notifications</code>	The channel and owner DM for each classification, and the ops alert channel
<code>company_fit</code>	Employee thresholds, target industries, disqualifiers
<code>personas</code>	Title patterns and the angle for each buying persona
<code>urgency</code>	The Tier 1 triggers and the renewal window
<code>scoring</code>	Points per component and the maximum
<code>competitors</code>	Named vendors, domains and the possible-competitor language tokens
<code>llm</code>	Model, prompt version and the confidence floor the AI must clear

Competitor identities live in their own table, `oak_competitors`, one row per vendor with its domains and aliases. Set `active` to false to stop a vendor matching.

Re-running `00` reseeds the policy row, so make live changes in the table rather than by rerunning setup.

How a scan flows

1. The booth app POSTs the badge scan to `01`.
2. `01` validates it, blocks replays, loads the policy, and classifies deterministically.
3. It answers the scanner in about two seconds with a provisional decision, then keeps working.
4. `02A` reads CRM context, `04` enriches the company and person, `05` adds bounded AI synthesis.

5. A final deterministic gate settles the tier.
6. 02 writes the contact, company and booth history to HubSpot.
7. 03 posts the card to the channel the policy names for that classification.
8. A rep taps a button; 06 verifies Slack's signature, writes to HubSpot, and replies in thread.